BrokeP — Software Requirements & Architecture Doc

Version: MVP 1.0 (Structure Rev 2) Status: Superseeds original SRS. Structure below is final — build against this.

---

1. Purpose

BrokeP is a lightweight offline desktop app for an independent stock broker to replace Excel-based trade ledgers. It records trades, tracks client cash balances, computes brokerage/P&L, and produces statements.

This doc exists to lock the **project structure** and the **data model** before more UI code gets written on top of a shaky schema. Everything below maps 1:1 to the folder tree in your screenshot.

---

2. Project Structure (as established)

```
BROKEP_1.0/
├── app/
│   ├── database/
│   │   ├── repositories/
│   │   │   ├── cash_repository.py
│   │   │   ├── client_repository.py
│   │   │   └── trade_repository.py
│   │   ├── connection.py
│   │   └── schema.py
│   ├── domain/
│   │   ├── calculations/
│   │   ├── entities/
│   │   └── validators/
│   ├── exports/
│   ├── services/
│   ├── ui/
│   └── utils/
├── assets/
│   ├── fonts/
│   ├── icons/
│   └── images/
├── data/
│   └── backups/
```

```
├── docs/
│   └── BUSINESS_RULES.md
├── tests/
└── README.md
```

Layer responsibilities (strict, don't blur these)

Layer	Folder	Allowed to do	NOT allowed to do
Repository	`database/repositories/`	Raw SQL, parameterized queries, CRUD	Business logic, validation, P/L math
Domain — entities	`domain/entities/`	Plain dataclasses mirroring DB rows	DB access, UI
Domain — calculations	`domain/calculations/`	Pure functions: gross value, brokerage, P/L, running balance	DB access, UI, side effects
Domain — validators	`domain/validators/`	Field presence/type/range checks, raise on bad input	DB access, UI
Service	`services/`	Orchestrate repos + domain calcs + validators into one use-case	Raw SQL, tkinter widgets
UI	`ui/`	Widgets, layout, user input capture, calling services	Raw SQL, P/L formulas
Exports	`exports/`	PDF/Excel generation from service output	Business math, DB writes
Utils	`utils/`	Date formatting, path helpers, generic helpers	Anything domain-specific

If you ever find yourself writing a `SELECT` inside `services/` or a formula inside `ui/`, that's the signal you've broken a layer — move it down.

---

3. The Core Structural Fix: Trade Lifecycle

This is the thing that was broken in the old flat `ledger` table. Read this section carefully before touching `trade_repository.py`.

Problem with old design: every BUY and SELL was one independent row. There was no way to know which SELL closed which BUY, so "close a trade" and "P/L report" (FR-07, FR-08, FR-09) were structurally impossible.

Fix: a `trades` row represents **one full trade**, not one execution. It starts `OPEN` on entry and gets `exit_*` fields filled in when closed.

```
Client places order
        ↓
Broker executes → INSERT trades (status=OPEN, exit fields NULL)
        ↓
Trade sits OPEN, visible in "Open Trades"
        ↓
Client exits → UPDATE trades SET exit_price, exit_date, status=CLOSED
        ↓
gross_pl / net_pl computed at close time, stored (not recomputed live)
        ↓
Client balance updated by net_pl
```

Rule: **a CLOSED trade never returns to OPEN.** No delete/reopen path in MVP.

Cash deposits/withdrawals are a *separate* entity (`cash_transactions`) — they do not go through the `trades` table. Don't merge them just because both eventually touch "balance."

---

4. Data Model

4.1 `clients` (client_repository.py)

Column	Type	Notes
client_id	INTEGER PK AUTOINCREMENT	
name	TEXT NOT NULL	
phone	TEXT	
email	TEXT	

Column	Type	Notes
pan	TEXT	optional
created_at	TEXT	ISO date, set on insert
is_active	INTEGER	1/0, soft-delete flag

Balance is **not** stored here — it's derived (see §6, Running Balance) or cached in a separate `client_balances` row updated transactionally. Pick one; recommend deriving on read for MVP, cache later if performance demands it.

4.2 `trades` (trade_repository.py)

Column	Type	Notes
trade_id	INTEGER PK AUTOINCREMENT	
client_id	INTEGER FK → clients	
segment	TEXT	EQUITY / FNO / COMMODITY
symbol	TEXT NOT NULL	
quantity	INTEGER NOT NULL	
entry_date	TEXT NOT NULL	
entry_price	REAL NOT NULL	
entry_brokerage	REAL NOT NULL	
status	TEXT NOT NULL	'OPEN' or 'CLOSED'
exit_date	TEXT	NULL until closed
exit_price	REAL	NULL until closed
exit_brokerage	REAL	NULL until closed
service_fee	REAL DEFAULT 0	
gross_pl	REAL	NULL until closed, computed once
net_pl	REAL	NULL until closed, computed once
remarks	TEXT	

4.3 `cash_transactions` (cash_repository.py)

Column	Type	Notes
txn_id	INTEGER PK AUTOINCREMENT	
client_id	INTEGER FK → clients	
txn_date	TEXT NOT NULL	
txn_type	TEXT NOT NULL	DEPOSIT / WITHDRAWAL / ADJUSTMENT
amount	REAL NOT NULL	always positive; sign handled by txn_type
remarks	TEXT	

`schema.py` owns all three `CREATE TABLE IF NOT EXISTS` statements. `connection.py` owns the single `get_connection()` used everywhere — no repository opens a raw `sqlite3.connect()` itself.

---

5. Domain Layer

5.1 `domain/entities/`

Plain dataclasses, one file per entity: `client.py`, `trade.py`, `cash_transaction.py`. These are what repositories return and services pass around — never raw tuples from `cursor.fetchall()`.

5.2 `domain/calculations/`

Pure functions, no I/O. This replaces the old `finance_engine.py`, split up:

- `gross_value(price, quantity) -> float`
- `brokerage(gross_value, manual_override=None) -> float`
- `gross_pl(entry_price, exit_price, quantity) -> float` — FR-08
- `net_pl(gross_pl, entry_brokerage, exit_brokerage, service_fee) -> float` — FR-09
- `running_balance(previous, deposits, withdrawals, net_pl, adjustments) -> float`

5.3 `domain/validators/`

One validator per entity, raising `ValueError` with a clear message on failure — services

catch and surface to UI. Covers FR-03 (mandatory fields before saving): required fields present, quantity > 0, price > 0, valid txn_type/segment enums, client exists before trade insert.

---

6. Calculations Reference

```
Gross Value        = Quantity × Price
Gross P/L           = (Exit Price − Entry Price) × Quantity
Net P/L             = Gross P/L − Entry Brokerage − Exit Brokerage − Service
Fee
Running Balance     = Previous Balance + Deposits − Withdrawals + Net P/L ±
Adjustments
```

---

7. Services Layer

One service module per use-case cluster, each calling validators → calculations → repositories in that order:

- `client_service.py` — create/edit/search/view client (FR-01)
- `trade_service.py` — `open_trade()`, `close_trade()` (FR-02, FR-04–07)
- `cash_service.py` — `deposit()`, `withdraw()`, `adjust()` (FR-11, FR-12)
- `report_service.py` — assembles data for daily P/L, client statement, open/closed trade lists (FR-13–15), hands off to `exports/`

---

8. Functional Requirements (renumbered against new structure)

ID	Requirement
FR-01	System shall create/edit/search/view a client.
FR-02	System shall record a new trade as status=OPEN.
FR-03	System shall validate mandatory fields before any save (domain/validators).
FR-04	System shall calculate Gross Value on trade entry.
FR-05	System shall calculate brokerage on entry and exit independently.

ID	Requirement
FR-06	System shall support a manual brokerage override per trade.
FR-07	System shall close an OPEN trade only (never re-open a CLOSED one).
FR-08	System shall calculate Gross P/L on close.
FR-09	System shall calculate Net P/L on close.
FR-10	System shall update client running balance after every trade close or cash transaction.
FR-11	System shall record cash deposits.
FR-12	System shall record cash withdrawals and manual adjustments.
FR-13	System shall generate a client statement (trades + cash, chronological).
FR-14	System shall list OPEN trades.
FR-15	System shall list CLOSED trades.
FR-16	System shall export statements/reports to PDF and Excel (`exports/`).

9. Non-Functional Requirements

- Fully offline, local SQLite only.
- Startup under 5 seconds.
- Handle thousands of trade rows without UI lag.
- All writes wrapped in transactions — no partial saves on crash.
- `data/backups/` — manual or on-close backup copy of `ledger.db` (simple file copy is enough for MVP).
- Graceful handling of invalid input at the validator layer, never a raw traceback in the UI.
- Packageable via PyInstaller.

10. Explicitly Out of Scope (MVP)

Cloud sync, multi-user, GST calculation, Excel import, automatic FIFO/lot matching, partial exits, margin calculation, dashboard analytics. Partial exits in particular will eventually

break the "one row = one trade" model above — when you get there, it needs its own design pass, don't bolt it on.

---

11. Build Order Recommendation

1. `database/schema.py` + `database/connection.py` — get the three tables right first.
2. `database/repositories/*.py` — pure CRUD against those tables.
3. `domain/entities/`, `domain/calculations/`, `domain/validators/` — no DB dependency, easiest to unit test in isolation (`tests/`).
4. `services/*.py` — wire the above together.
5. `ui/` — already mostly built; adjust calls to hit services instead of the old flat ledger functions.
6. `exports/` — last, once report_service output shape is stable.

Don't start on `ui/` polish or `exports/` until `trades` schema + close-trade service is working end-to-end — that's the part everything else depends on.